# Busca Linear e Busca Binária

## 1. O que é uma busca

**Busca** é o processo de encontrar, a partir de um critério específico, uma informação particular dentro de uma coleção de dados. Neste material, o termo será usado em um sentido mais restrito: encontrar um **item específico** dentro de uma estrutura de dados do tipo **sequencial**, como as listas que já conhecemos.

Buscar (e ordenar) são os problemas mais comuns da computação: aparecem desde achar um contato no celular até localizar um pedido em uma loja virtual. Existem vários métodos de busca, e a escolha do mais adequado depende de:

- **quantidade de dados:** procurar em 10 itens é diferente de procurar em 10 milhões;
- **ordenação inicial dos dados:** a lista está em ordem (crescente, por exemplo) ou está embaralhada?
- **inserções e retiradas:** a lista recebe novos itens e perde itens o tempo todo, ou é quase sempre a mesma?

Estudaremos os dois métodos clássicos: a **busca linear** (também chamada de busca sequencial) e a **busca binária**. Nas duas, a função **devolve o índice** (a posição) do item encontrado, ou `-1` quando o item não existe na lista.

## 2. Buscas que o Python já oferece

O Python já traz formas prontas de procurar um item em uma lista:

```
lista = [20, 5, 15, 23, 67, 45, 1, 76]

print(lista.index(23))      # 3     (posição do 23 na lista)
print(23 in lista)          # True  (o 23 existe na lista?)
print(99 in lista)          # False
```

- `lista.index(item)` devolve a **posição** do item. Se o item não existir, ela **não** devolve `-1`: ela lança `ValueError` (a exceção da Aula 08). Para usá-la com segurança, é preciso tratar o erro:

  ```
  try:
      posicao = lista.index(99)
  except ValueError:
      posicao = -1
  ```

- `item in lista` devolve apenas `True` ou `False`, sem dizer a posição.
- Nos textos (strings), o método `find` devolve a posição ou `-1` quando não encontra, igual ao que vamos implementar: `"banana".find("n")` vale `2` e `"banana".find("z")` vale `-1`.

Por baixo dos panos, **as três usam a busca linear**: comparam o item com os valores da lista, um por um. Elas funcionam bem, mas não conhecem a ordem dos dados. Quando a lista é muito grande e está em ordem, é possível fazer buscas **muito mais rápidas**, e é isso que vamos implementar. Escrever os algoritmos nós mesmos também permite adaptá-los: contar comparações, alterar o item encontrado, devolver todas as posições etc. (exercícios desta aula).

## 3. Busca linear

A **busca linear** é o algoritmo de busca mais simples. Ela pode ser executada em uma lista **não ordenada**. É também a técnica usada por `index`, `in` e `find`.

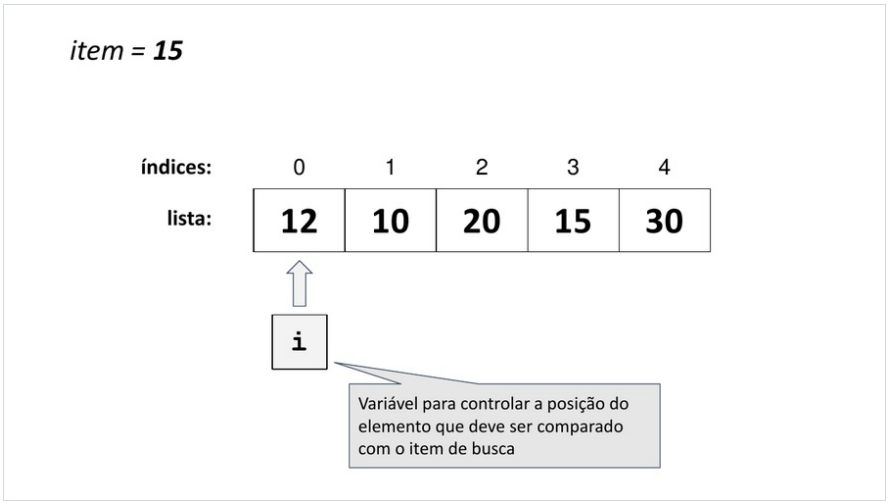
A ideia: percorrer a lista comparando o item procurado com o valor de cada posição.

1. Começa na posição `0`.
2. Se o valor da posição for igual ao item, **devolve essa posição** e termina.
3. Se não for, passa para a próxima posição.
4. Se a lista toda foi percorrida e o item não apareceu, **devolve** `-1`.

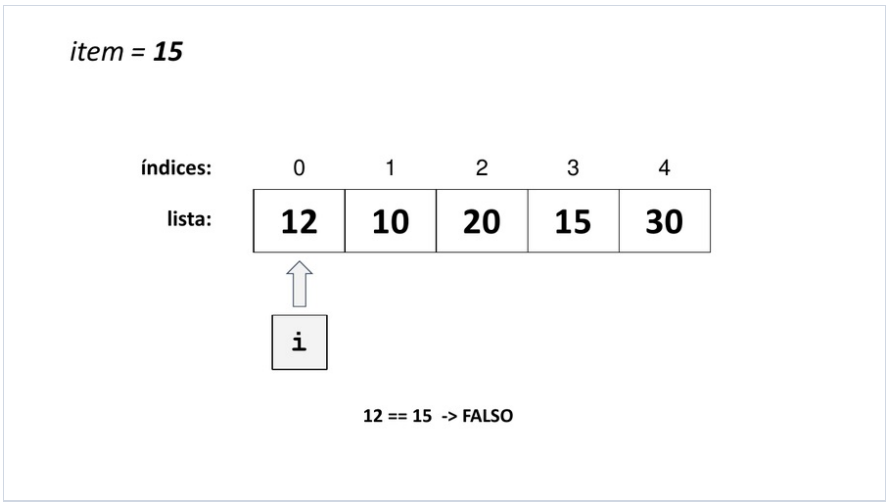
### 3.1 Exemplo 1: o item existe (`item = 15`)

Vamos procurar o `15` na lista `[12, 10, 20, 15, 30]`. Os números de cima são os **índices** (as posições) e os de baixo são os **valores**. A variável `i` guarda a posição que está sendo comparada neste momento.

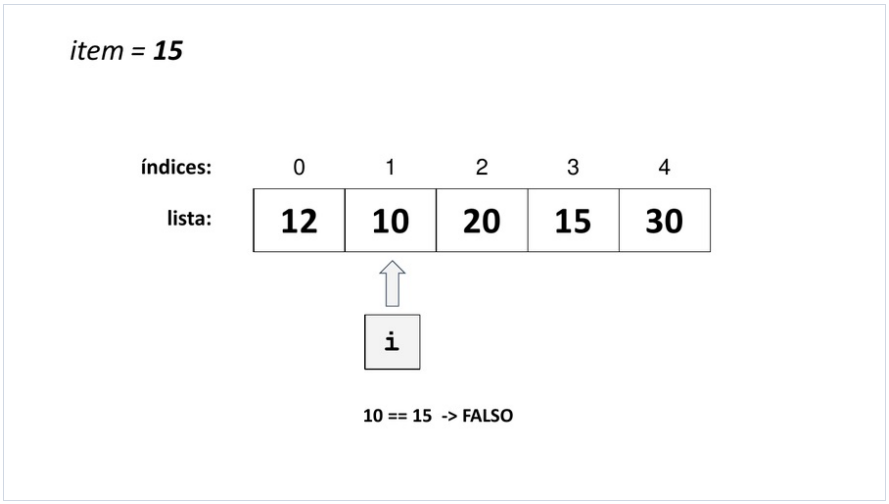
**Passo 1: começa na posição 0.** A variável `i` aponta para o índice `0`.



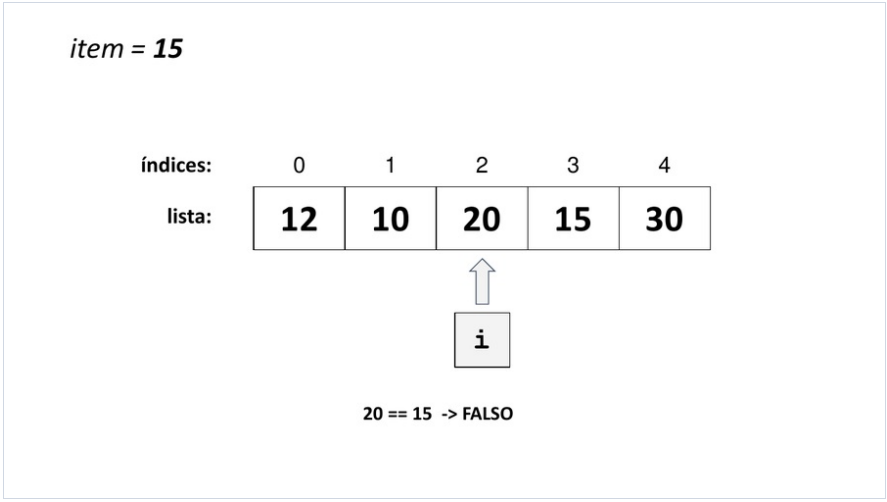

**Passo 2:** `i = 0`**.** Compara `lista[0]` com o item: `12 == 15`? Falso. Passa para a próxima posição.



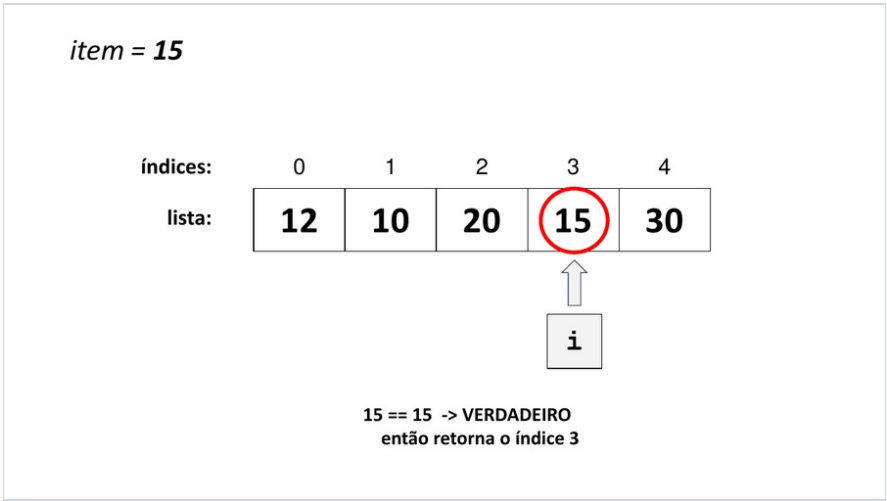

**Passo 3:** `i = 1`**.** `10 == 15`? Falso. Próxima posição.



**Passo 4:** `i = 2`. `20 == 15`? Falso. Próxima posição.




**Passo 5:** `i = 3`. `15 == 15`? **Verdadeiro.** O item foi encontrado, então a função devolve o índice `3` e termina. Os valores que vinham depois (o `30`) nem chegam a ser comparados.



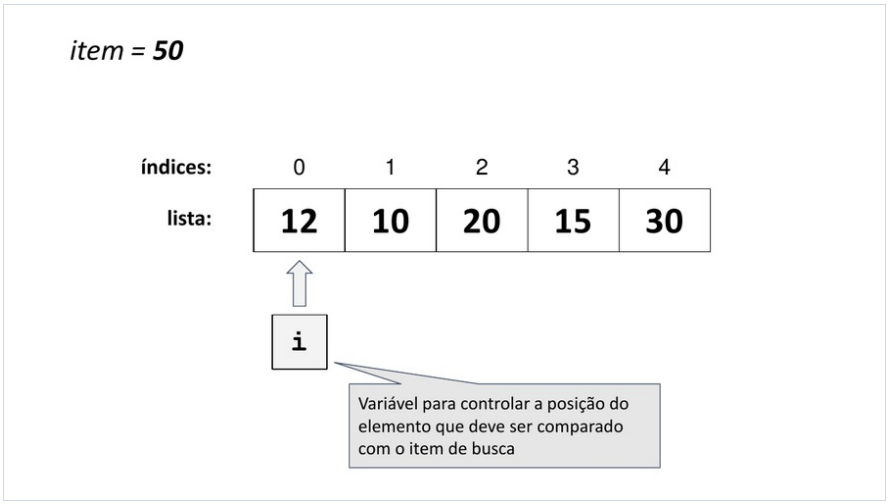

Resumo da execução:

| Passo | `i` | Comparação | Resultado | O que acontece |
|---|---|---|---|---|
| 1 | 0 | `12 == 15` | Falso | Segue para a próxima posição |
| 2 | 1 | `10 == 15` | Falso | Segue para a próxima posição |
| 3 | 2 | `20 == 15` | Falso | Segue para a próxima posição |
| 4 | 3 | `15 == 15` | **Verdadeiro** | Devolve `3` e termina (4 comparações) |

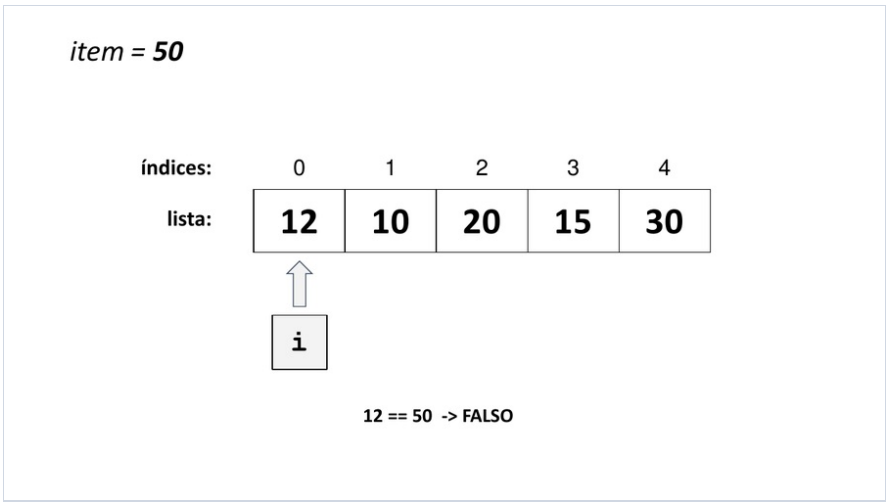
## 3.2 Exemplo 2: o item não existe (`item = 50`)

Agora vamos procurar o `50` na mesma lista `[12, 10, 20, 15, 30]`.

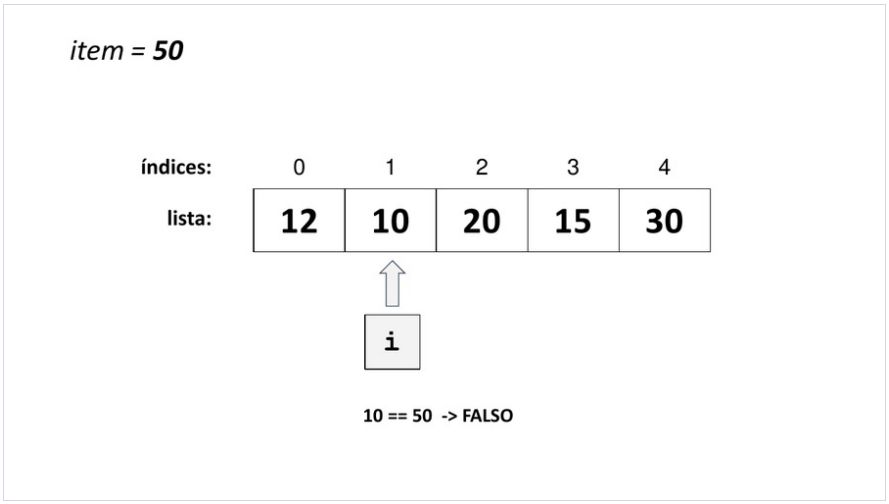
**Passo 1: começa na posição 0.**



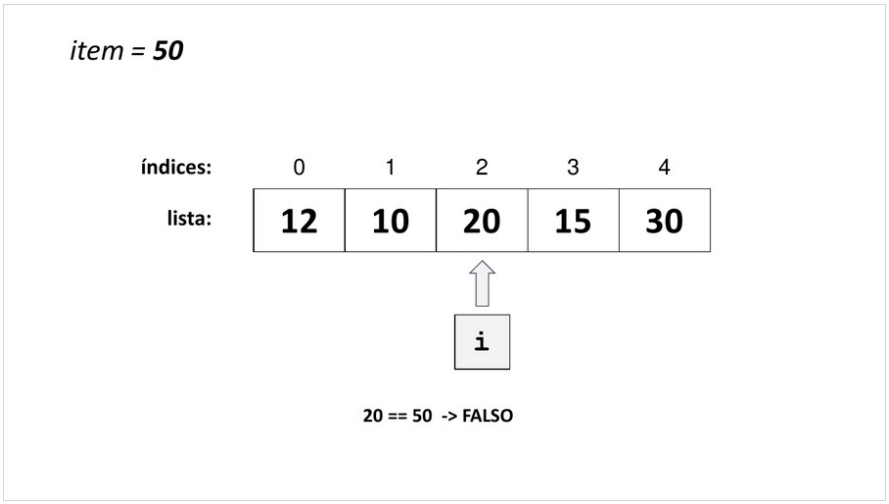

**Passo 2:** `i = 0.` `12 == 50`? Falso.



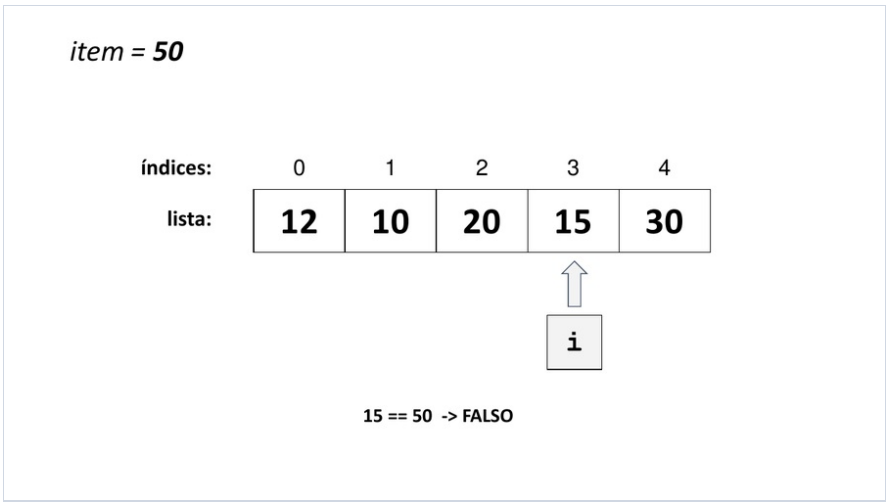

**Passo 3:** `i = 1.` `10 == 50`? Falso.



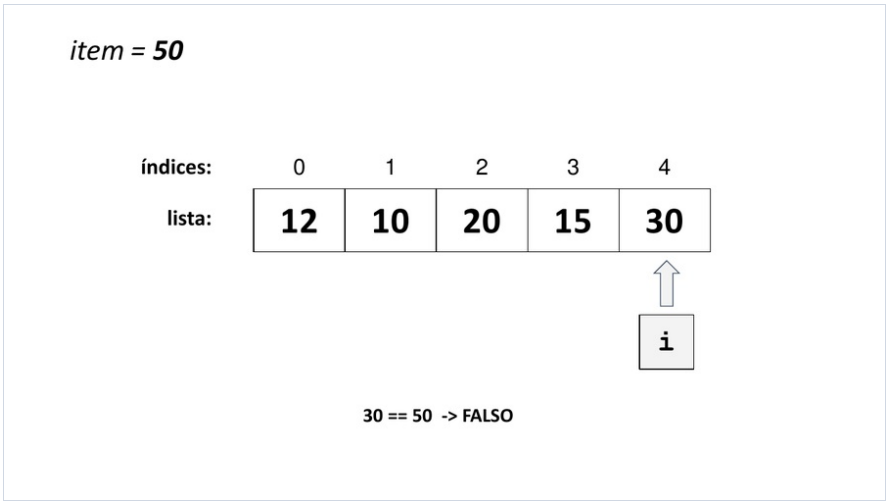
**Passo 4:** `i = 2.` `20 == 50`? Falso.



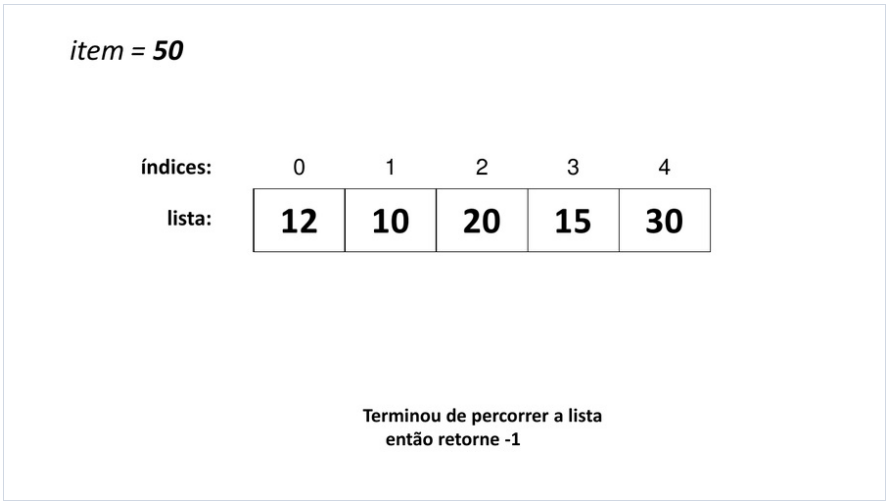

**Passo 5:** `i = 3.` `15 == 50`? Falso.




**Passo 6:** `i = 4.` `30 == 50`? Falso. Era a última posição.



**Passo 7: fim da lista.** Não há mais posições para comparar. Só agora é possível afirmar que o item **não existe**, e a função devolve `-1`.




| Passo | `i` | Comparação | Resultado | O que acontece |
|---|---|---|---|---|
| 1 | 0 | `12 == 50` | Falso | Segue |
| 2 | 1 | `10 == 50` | Falso | Segue |
| 3 | 2 | `20 == 50` | Falso | Segue |
| 4 | 3 | `15 == 50` | Falso | Segue |
| 5 | 4 | `30 == 50` | Falso | Segue |
| 6 | fim | (lista acabou) | | Devolve `-1` (5 comparações) |

## 3.3 Implementação

```
def busca_linear(lista, item):
    for i in range(len(lista)):         # percorre lista do índice 0 a n-1
        if lista[i] == item:
            return i                    # encontrou o item na posição i
    return -1                           # percorreu tudo e não encontrou
```

Linha a linha:

- `range(len(lista))` gera os índices `0, 1, 2, ..., n-1`, e `i` recebe um deles a cada volta do `for`.
- `lista[i] == item` é a **comparação** do passo a passo acima.
- `return i` **encerra a função** na hora, devolvendo a posição. É por isso que, no exemplo 1, o `30` nunca foi comparado.
- `return -1` está **fora** do `for` (note o recuo): só chega nessa linha quem percorreu a lista inteira sem nunca executar o `return i`.

```
lista = [12, 10, 20, 15, 30]
print(busca_linear(lista, 15))      # 3
print(busca_linear(lista, 50))      # -1
```

## 3.4 Pontos de atenção

**1) O `return -1` dentro do laço é um erro muito comum.** Veja esta versão:

```
def busca_linear_errada(lista, item):
    for i in range(len(lista)):
        if lista[i] == item:
            return i
        else:
            return -1       # ERRO: decide "não achou" logo na primeira comparação
```

Veja `busca_linear_errada([12, 10, 20, 15, 30], 15)` passo a passo:

| Passo | `i` | Comparação | O que acontece |
|---|---|---|---|
| 1 | 0 | `12 == 15` é falso | Cai no `else` e executa `return -1`: **a função termina aqui** |

O resultado é `-1`, embora o `15` esteja no índice 3. A lista inteira precisa ser examinada antes de afirmar que o item não existe. Por isso o `return -1` fica depois do laço.

**2) Por que `-1` e não `0`?** Porque `0` é uma posição válida (o primeiro item). Um valor que **nunca** poderia ser índice de um item encontrado indica "não achei". Mas cuidado: no Python, `-1` também é um índice que funciona, pois `lista[-1]` é o **último** item da lista. Usar o resultado da busca sem conferir causa um erro silencioso:

```
lista = [12, 10, 20, 15, 30]
posicao = busca_linear(lista, 50)       # -1: o 50 não existe
print(lista[posicao])                   # 30  (o último item, sem nenhum erro!)
```

Por isso, sempre teste o resultado antes de usá-lo:

```
posicao = busca_linear(lista, 50)
if posicao != -1:
    print(f"Encontrado na posição {posicao}")
else:
    print("Item não encontrado")
```

### 3.5 Desempenho da busca linear

O custo de uma busca é medido pela **quantidade de comparações**.

- **Melhor caso:** o item é o primeiro da lista. Uma comparação.
- **Pior caso:** o item é o **último** ou **não existe**. São `N` comparações, em que `N` é o tamanho da lista.

Ou seja, a quantidade de comparações é **proporcional ao tamanho da lista**: se a lista dobra de tamanho, o pior caso dobra também.

| Itens na lista | Comparações no pior caso |
|---|---|
| 10 | 10 |
| 1.000 | 1.000 |
| 1.000.000 | 1.000.000 |

## 4. Busca binária

A **busca binária** é um algoritmo um pouco mais sofisticado. Ela só pode ser executada em listas **previamente ordenadas**. Em troca dessa exigência, ela é muito mais rápida.

Um exemplo do dia a dia: você abre um dicionário impresso para procurar a palavra "pêssego". Você não começa na primeira página e lê uma por uma. Você abre mais ou menos no meio, vê que ali está a letra "M", conclui que "pêssego" vem depois, e descarta toda a primeira metade. Repete o processo com a metade que sobrou. Outro exemplo é o jogo de adivinhar um número de 1 a 100: se você chuta 50 e ouvir "é maior", já eliminou 50 possibilidades de uma vez.

O algoritmo:

1. Encontra a posição do **meio** da lista.
2. Compara o item com o valor da posição do meio:
   - se for **igual**, devolve essa posição e termina;
   - se o item for **maior** que o valor do meio, o item só pode estar na **segunda metade**: repete o processo considerando apenas ela;
   - se o item for **menor** que o valor do meio, o item só pode estar na **primeira metade**: repete o processo considerando apenas ela.
3. Repete até que o item seja encontrado ou não sobre mais nenhum elemento para analisar (então devolve `-1`).

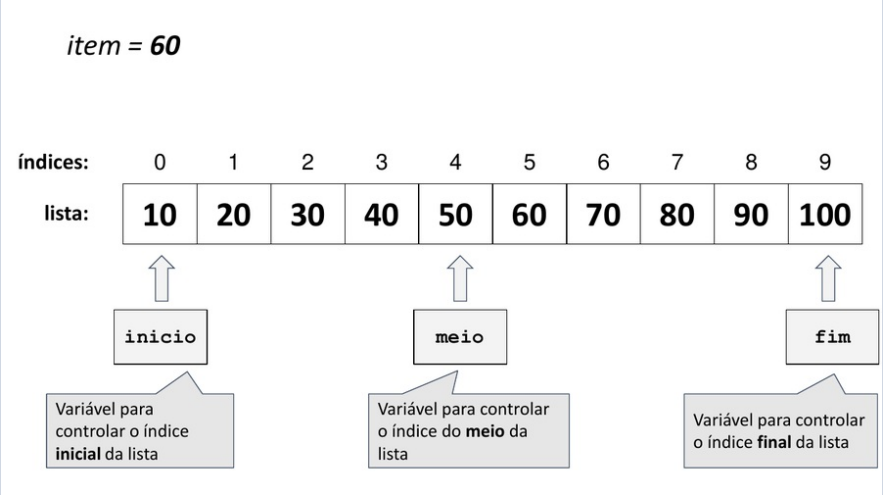
Para controlar qual parte da lista ainda está valendo, usamos três variáveis:

- `inicio`: índice do primeiro elemento da parte que ainda será analisada;
- `fim`: índice do último elemento dessa parte;
- `meio`: índice do elemento do meio dessa parte, calculado por `meio = (inicio + fim) // 2`. O `//` é a **divisão inteira** (sem casas decimais): `(0 + 9) // 2` vale `4`, e não `4.5`.

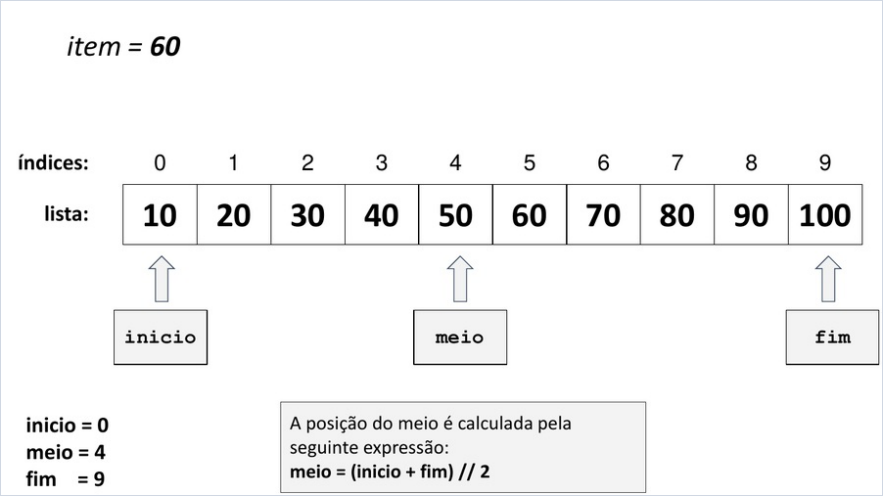
### 4.1 Exemplo 1: o item existe (`item = 60`)

Vamos procurar o `60` na lista ordenada `[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]` (índices de `0` a `9`).

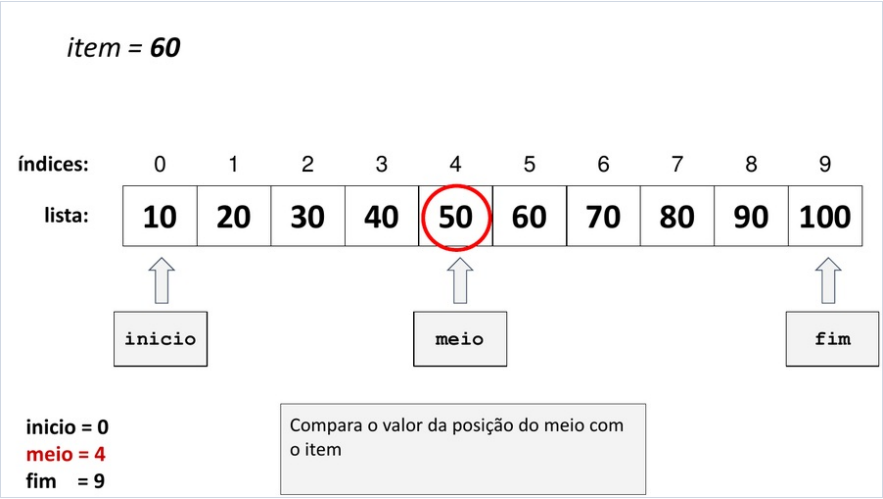
**Passo 1: as três variáveis.** No começo, `inicio` aponta para o primeiro elemento e `fim` para o último. O `meio` fica entre eles.



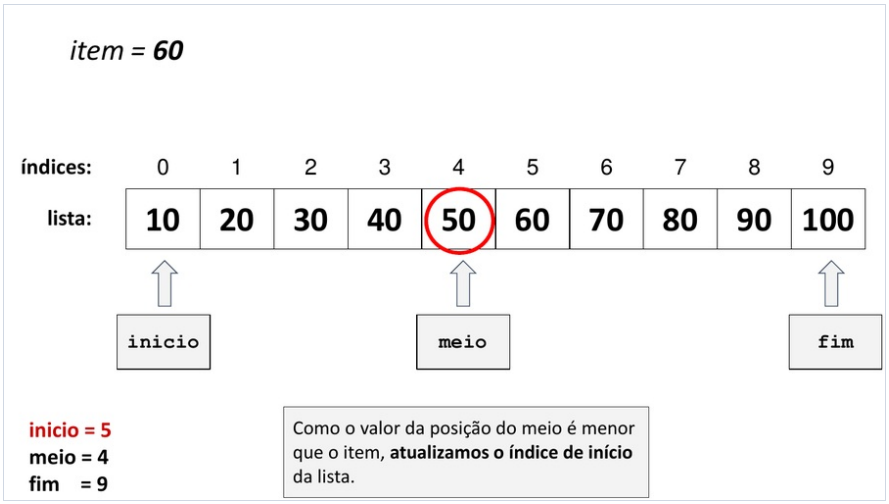

**Passo 2: calculando o meio.** Com `inicio = 0` e `fim = 9`, temos `meio = (0 + 9) // 2 = 4`.




**Passo 3: comparando com o meio.** O valor da posição `4` é `50`. Compara `50` com o item `60`: `50` é diferente de `60`.



**Passo 4: descartando a primeira metade.** Como `50 < 60`, o `60` só pode estar **depois** do meio. Tudo de `0` até `4` é descartado: `inicio = meio + 1 = 5`.



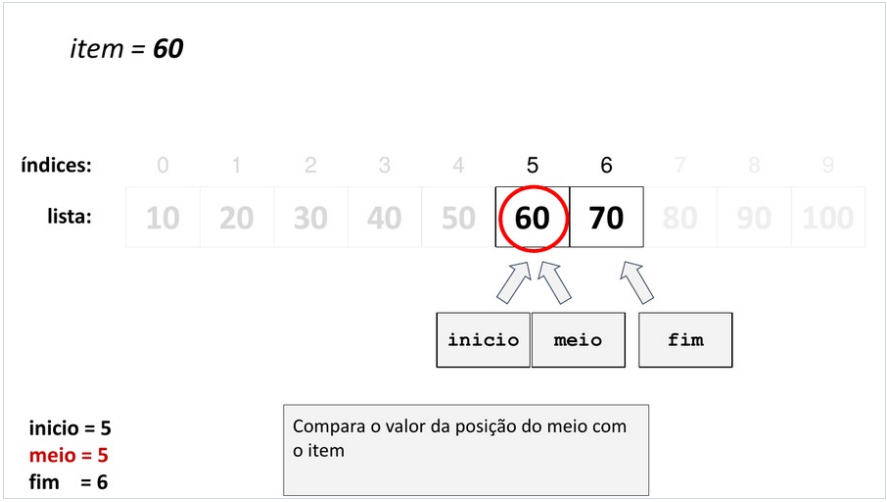

**Passo 5: novo meio.** Agora `inicio = 5` e `fim = 9`, então `meio = (5 + 9) // 2 = 7`. O valor da posição `7` é `80`, diferente de `60`.




**Passo 6: descartando a segunda metade.** Como `80 > 60`, o `60` só pode estar **antes** do meio. Tudo de `7` até `9` é descartado: `fim = meio - 1 = 6`.



**Passo 7: novo meio.** Agora `inicio = 5` e `fim = 6`, então `meio = (5 + 6) // 2 = 5`. O valor da posição `5` é `60`.




**Passo 8: encontrou.** `60 == 60`: o item foi encontrado e a função devolve a posição `5`.




Resumo da execução:

| Rodada | `inicio` | `fim` | `meio` | `lista[meio]` | Comparação com `60` | O que acontece |
|---|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 50 | `50 < 60` | Descarta a primeira metade: `inicio = 5` |
| 2 | 5 | 9 | 7 | 80 | `80 > 60` | Descarta a segunda metade: `fim = 6` |
| 3 | 5 | 6 | 5 | 60 | `60 == 60` | **Encontrou:** devolve `5` |

Foram só **3 comparações**. A busca linear precisaria de 6 (passando pelo `10`, `20`, `30`, `40`, `50` até chegar no `60`).

## 4.2 Exemplo 2: o item não existe (`item = 55`)

Na mesma lista, vamos procurar o `55`, que não existe:

| Rodada | `inicio` | `fim` | `meio` | `lista[meio]` | Comparação com `55` | O que acontece |
|---|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 50 | `50 < 55` | Descarta a primeira metade: `inicio = 5` |
| 2 | 5 | 9 | 7 | 80 | `80 > 55` | Descarta a segunda metade: `fim = 6` |
| 3 | 5 | 6 | 5 | 60 | `60 > 55` | Descarta a segunda metade: `fim = 4` |
| 4 | 5 | 4 | | | | `inicio (5)` passou de `fim (4)`: **não sobrou nenhum elemento**. Devolve `-1` |

A busca termina quando `inicio` **ultrapassa** `fim`: isso significa que a parte da lista que ainda poderia conter o item ficou vazia.

## 4.3 Implementação

```
def busca_binaria(lista, item):
    inicio = 0
    fim = len(lista) - 1
    while inicio <= fim:
        meio = (inicio + fim) // 2      # índice da metade da lista
        if lista[meio] == item:         # encontrou o item
            return meio
        elif item > lista[meio]:        # busca na segunda metade
            inicio = meio + 1
        else:                           # busca na primeira metade
            fim = meio - 1
    return -1                           # não encontrou o item
```

Linha a linha:

- `inicio = 0` e `fim = len(lista) - 1`: no começo, a parte analisada é a lista inteira.
- `while inicio <= fim`: repete **enquanto ainda sobrar algum elemento** para analisar.
- `meio = (inicio + fim) // 2`: calcula o meio da parte atual.
- Se `lista[meio] == item`: achou, devolve o índice.
- Se `item > lista[meio]`: o item está na segunda metade, então `inicio = meio + 1` (o `meio` já foi comparado, por isso o `+ 1`).
- Caso contrário (`item` é menor): está na primeira metade, então `fim = meio - 1`.
- `return -1`: só chega aqui se o laço terminou, ou seja, se `inicio` passou de `fim` sem que o item aparecesse.

```
lista = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
print(busca_binaria(lista, 60))     # 5
print(busca_binaria(lista, 55))     # -1
```

## 4.4 Pontos de atenção

**1) A lista PRECISA estar ordenada.** A busca binária descarta metade da lista **supondo** que os valores estão em ordem. Veja o que acontece em uma lista embaralhada, procurando o `10` em `[30, 10, 20, 50, 40]` (o `10` existe, no índice 1):

| Rodada | `inicio` | `fim` | `meio` | `lista[meio]` | Comparação com `10` | O que acontece |
|---|---|---|---|---|---|---|
| 1 | 0 | 4 | 2 | 20 | `20 > 10` | Descarta a segunda metade: `fim = 1` |
| 2 | 0 | 1 | 0 | 30 | `30 > 10` | Descarta a segunda metade: `fim = -1` |
| 3 | 0 | -1 | | | | `inicio` passou de `fim`: devolve `-1` |

Na rodada 2, a busca viu o `30` na posição `0` e concluiu: "o `10` é menor, então estaria **antes** do `30`". Mas não existe nada antes da posição `0`. Como a lista estava fora de ordem, o `10` estava **depois** do `30`, e foi descartado por engano. O resultado `-1` está errado e o Python não avisa. Já a busca linear acharia o `10` sem problemas.

**2) Por que `inicio <= fim` e não `inicio < fim`?** Quando `inicio` e `fim` são iguais, ainda **resta um elemento** para conferir. Veja a busca do `7` na lista `[7]` (um único item), com o laço `while inicio < fim`:

| Passo | `inicio` | `fim` | `inicio < fim`? | O que acontece |
|---|---|---|---|---|
| 1 | 0 | 0 | Não (0 não é menor que 0) | O laço **nem começa** e devolve `-1`, mesmo o `7` estando lá |

Com `inicio <= fim`, o laço roda: `meio = 0`, `lista[0] == 7`, devolve `0`.

**3) Por que `meio + 1` e `meio - 1`?** O `meio` já foi comparado e não é o item. Se escrevermos `inicio = meio` em vez de `inicio = meio + 1`, a busca pode **nunca terminar**. Veja a busca do `20` na lista `[10, 20]` com `inicio = meio`:

| Rodada | `inicio` | `fim` | `meio` | `lista[meio]` | O que acontece |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 10 | `20 > 10`: `inicio = meio`, que vale `0` (nada mudou!) |
| 2 | 0 | 1 | 0 | 10 | Mesma situação da rodada 1 |
| ... | ... | ... | ... | ... | A parte analisada **nunca diminui**: laço infinito |

Com `inicio = meio + 1`, o `inicio` vira `1`, o próximo meio é `1`, e o `20` é encontrado. É a mesma regra da recursividade (Aula 12): a cada passo, o problema precisa ficar **menor**.

**4) Divisão inteira.** Se escrevermos `meio = (inicio + fim) / 2`, o resultado é um número decimal (`4.5`, por exemplo), e `lista[4.5]` causa `TypeError: list indices must be integers or slices, not float`. Por isso o operador é `//`.

## 4.5 Desempenho da busca binária

Na busca binária, a cada comparação **descartamos metade** da lista. O número de comparações corresponde a **quantas vezes conseguimos dividir a lista ao meio** até não sobrar nada.

Uma lista com 7 itens:

| Divisão | Cálculo | Resultado |
|---|---|---|
| 1 | `7 / 2` | 3,5 |
| 2 | `3,5 / 2` | 1,75 |
| 3 | `1,75 / 2` | 0,875 |

Em no máximo **3 divisões** a lista "termina". Uma lista com mil itens:

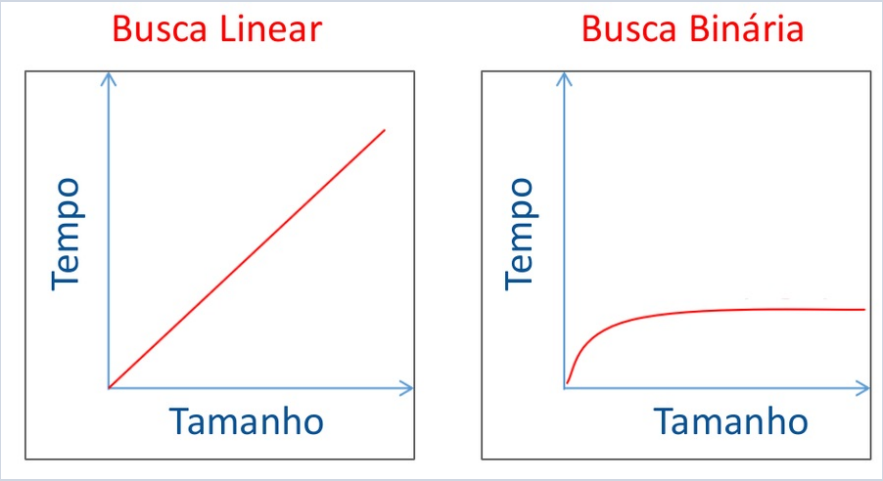
| Divisão | Cálculo | Resultado |
|---|---|---|
| 1 | `1000 / 2` | 500 |
| 2 | `500 / 2` | 250 |
| 3 | `250 / 2` | 125 |
| 4 | `125 / 2` | 62,5 |
| 5 | `62,5 / 2` | 31,25 |
| 6 | `31,25 / 2` | 15,625 |
| 7 | `15,625 / 2` | 7,8125 |
| 8 | `7,8125 / 2` | 3,90625 |
| 9 | `3,90625 / 2` | 1,953125 |
| 10 | `1,953125 / 2` | 0,9765625 |

Em no máximo **10 divisões** a lista "termina": mil itens exigem, no máximo, 10 comparações. Comparando com a busca linear:

| Itens na lista | Máximo de comparações: busca linear | Máximo de comparações: busca binária |
|---|---|---|
| 10 | 10 | 4 |
| 1.000 | 1.000 | 10 |
| 1.000.000 | 1.000.000 | 20 |
| 1.000.000.000 | 1.000.000.000 | 30 |

Cada vez que a lista **dobra** de tamanho, a busca binária faz apenas **1 comparação a mais**. Em uma lista de 1 bilhão de itens, ela confere no máximo 30. (Em matemática, esse número é o logaritmo na base 2 do tamanho da lista, arredondado para baixo, mais 1.)

## 5. Busca linear x busca binária




- O custo da **busca linear** é proporcional ao tamanho da lista: lista maior, tempo maior, na mesma proporção (a linha reta do gráfico).
- O custo de tempo da **busca binária** cresce muito lentamente (a curva do gráfico quase se achata). Por isso ela é a melhor opção para listas com grande quantidade de itens.

- Como **desvantagem**, a busca binária exige dados **ordenados**. Então é necessário analisar também o **custo de ordenar** os elementos.

| | **Busca linear** | **Busca binária** |
|---|---|---|
| Lista precisa estar ordenada? | Não | **Sim** |
| Comparações no pior caso | `N` | cerca de `log2(N)` (30 para 1 bilhão de itens) |
| Complexidade do código | Simples | Um pouco maior (`inicio`, `fim`, `meio`) |
| Boa para | Listas pequenas, listas desordenadas, uma única busca | Listas grandes e ordenadas, muitas buscas |

**Quando vale a pena ordenar para usar a busca binária?** Ordenar uma lista custa mais do que uma única busca linear. Veja uma lista com 1.000.000 de itens:

| **Situação** | **Busca linear** | **Busca binária** |
|---|---|---|
| 1 busca em uma lista desordenada | até 1.000.000 comparações | Ordenar primeiro (custo alto) e depois até 20 comparações: **não compensa** |
| 1.000 buscas na mesma lista | até 1.000 x 1.000.000 = 1.000.000.000 comparações | Ordena uma única vez e faz até 1.000 x 20 = 20.000 comparações: **compensa muito** |

Ou seja: se a lista será consultada **muitas vezes**, o custo de ordenar é pago uma única vez e as buscas seguintes são baratas. Se a lista muda o tempo todo (muitas inserções e retiradas), cada mudança pode bagunçar a ordem e exigir reordenar, o que reduz a vantagem da busca binária. (Os algoritmos de ordenação serão estudados em uma aula futura. Nesta aula, quando precisarmos de uma lista ordenada, usaremos a função `sorted()` que o Python já traz.)

Outro ponto: com **poucos itens**, a diferença é pequena e a busca linear, mais simples, resolve bem. Em uma lista de 6 itens, a busca binária pode até fazer mais comparações que a linear (acontece no projeto de hoje).

## 6. Exemplos de mercado

A ideia de "descartar metade a cada passo" aparece em muitos lugares:

- **Listas telefônicas e dicionários:** a página é escolhida abrindo perto do meio e conferindo para qual lado ir.
- **Bancos de dados:** os índices que aceleram consultas por nome, CPF ou número do pedido usam estruturas que seguem a mesma ideia de "dados ordenados + descartar metade".
- **Depuração de versões:** quando um programa passou a falhar em algum ponto de uma sequência de 1.000 versões, testa-se a versão do meio, descarta-se metade, e assim por diante. A ferramenta `git bisect` automatiza exatamente isso.
- **Lojas virtuais:** localizar um produto por nome, preço ou código em um catálogo ordenado, como faremos no projeto de hoje.

## 7. Tema complementar: medindo o tempo de execução com `datetime`

Dizer que uma busca é "mais rápida" fica mais convincente quando **medimos** o tempo. Para isso, o Python tem o módulo `datetime`, que trabalha com datas e horas (é uma biblioteca padrão: já vem instalada, só precisa ser importada).

```
        from datetime import datetime
```

- `datetime.now()` devolve a **data e hora exatas** do momento em que é chamada.
- Ao **subtrair** dois momentos (`final - inicial`), o resultado é a **duração** entre eles (um objeto chamado `timedelta`), que o `print` mostra no formato `horas:minutos:segundos.microssegundos`, como em `0:00:00.318419` (318 milésimos de segundo).
- O método `.total_seconds()` converte essa duração para um número decimal de **segundos** (`0.318419`), mais fácil de formatar e comparar.

A estratégia de medição é sempre a mesma: guardar a hora **antes**, executar o que se quer medir, guardar a hora **depois** e subtrair.

```python
from datetime import datetime


def busca_linear(lista, item):
    for i in range(len(lista)):
        if lista[i] == item:
            return i
    return -1


def busca_binaria(lista, item):
    inicio = 0
    fim = len(lista) - 1
    while inicio <= fim:
        meio = (inicio + fim) // 2
        if lista[meio] == item:
            return meio
        elif item > lista[meio]:
            inicio = meio + 1
        else:
            fim = meio - 1
    return -1


lista_numeros = list(range(10000000))       # lista com 10 milhões de itens (0 a 9999999)

tempo_inicial = datetime.now()
print(busca_binaria(lista_numeros, -40))
tempo_final = datetime.now()
print(f'Tempo Busca Binária: {tempo_final - tempo_inicial}')

tempo_inicial = datetime.now()
print(busca_linear(lista_numeros, -40))
tempo_final = datetime.now()
print(f'Tempo Busca Linear: {tempo_final - tempo_inicial}')
```

Exemplo de saída (os tempos mudam de um computador para outro, e de uma execução para outra):

```
-1
Tempo Busca Binária: 0:00:00.000028
-1
Tempo Busca Linear: 0:00:00.318419
```

O que o programa faz:

- `list(range(10000000))` cria a lista ordenada `[0, 1, 2, ..., 9999999]`. Ela está em ordem crescente, então serve para as duas buscas.
- O item procurado, `-40`, **não existe** na lista. É de propósito: é o **pior caso** da busca linear (compara os 10 milhões de itens antes de desistir) e o caso em que a busca binária faz mais comparações, ainda assim poucas (menos de 25).
- Neste exemplo, a busca binária levou cerca de 28 **milionésimos** de segundo, e a linear cerca de 0,3 segundo: mais de 10 mil vezes mais.

**Cuidado com o tamanho da lista.** Uma versão comum deste teste usa `list(range(500000000))`, com 500 milhões de itens. Cada número inteiro ocupa cerca de 28 bytes na memória, mais 8 bytes da referência dentro da lista. Isso dá algo perto de 18 gigabytes só para montar a lista, o que trava ou encerra o programa na maioria dos computadores. Com 10 milhões de itens (algumas centenas de megabytes), o teste roda bem e a diferença já é nítida.

(O módulo `datetime` serve para medições simples, como esta. Para medir tempos muito pequenos com mais precisão, o Python também oferece `time.perf_counter()`, do módulo `time`.)

---

## 8. Exercícios principais

Os exercícios abaixo estão em ordem **crescente de dificuldade**. Em todos, as listas de números inteiros usadas nas buscas binárias devem estar em **ordem crescente**.

### Exercício 1: Contando comparações na busca linear

Implemente a função `busca_linear_comparacoes(lista, item)`, alterando o algoritmo de busca linear para que a função retorne a **quantidade de comparações** realizadas na tentativa de encontrar um determinado item.

Por exemplo, para `lista = [12, 10, 20, 15, 30]`:

- `item = 15` (existe, no índice 3) devolve `4`;
- `item = 50` (não existe) devolve `5`.

### Exercício 2: Todas as posições de um item

Escreva a função `buscar_todas_posicoes(lista, item)`, que usa a ideia da busca linear para devolver uma **lista com todos os índices** em que o item aparece na lista, em ordem crescente. Se o item não aparecer, a função devolve uma lista vazia.

Por exemplo: `buscar_todas_posicoes([3, 5, 7, 3, 3, 6, 3], 3)` devolve `[0, 3, 4, 6]`.

Dica: diferente da busca da aula, aqui **não** dá para encerrar a função ao encontrar o primeiro. Pense onde guardar as posições achadas.

### Exercício 3: Contando comparações na busca binária

Implemente a função `busca_binaria_comparacoes(lista, item)`, alterando o algoritmo de busca binária para que a função retorne a **quantidade de comparações** realizadas na tentativa de encontrar um determinado item.

Considere que cada rodada do laço faz **uma** comparação (a do item com o valor do meio). Por exemplo, para `[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]`: `item = 60` devolve `3` e `item = 55` (não existe) também devolve `3`.

Depois, compare: quantas comparações a busca linear e a busca binária fazem para procurar o item `-45` em `list(range(10000))`?

### Exercício 4: Busca binária que altera o item encontrado

Implemente a função `busca_binaria_alteracao(lista, item, novo_valor)`, alterando o algoritmo de busca binária para que ele se torne um algoritmo de **atualização**. Caso o item seja encontrado, a função faz a alteração para um novo valor. Deve alterar **apenas a primeira ocorrência** do valor encontrado.

A função devolve o **índice que foi alterado**, ou `-1` se o item não existir (nesse caso, a lista não é modificada).

Por exemplo:

- `lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`; chamando `busca_binaria_alteracao(lista, 10, 20)`, a lista passa a ser `[1, 2, 3, 4, 5, 6, 7, 8, 9, 20]`.
- `lista = [1, 2, 2, 2, 3]`; chamando `busca_binaria_alteracao(lista, 2, 99)`, a lista passa a ser `[1, 99, 2, 2, 3]` (só a primeira ocorrência do `2`, no índice 1).

Dica: em uma lista com valores repetidos, a busca binária comum para na **primeira posição em que acha** o item, que não é necessariamente a primeira ocorrência. O que fazer, depois de achar, para conferir se existe uma ocorrência mais à esquerda?

### Exercício 5: Busca binária recursiva

Implemente uma versão **recursiva** da função de busca binária: `busca_binaria_recursiva(lista, item, inicio, fim)`, em que `inicio` e `fim` são os índices que delimitam a parte da lista que ainda será analisada. Para procurar na lista inteira, a primeira chamada recebe `inicio = 0` e `fim = len(lista) - 1`.

Dica (Aula 12): pense na condição de parada e em qual é o problema **menor** de cada chamada. Existem duas situações em que a função não precisa chamar a si mesma.

---

## 9. Exercício complementar

O exercício abaixo é **complementar**: liga o desempenho da busca binária (seção 4.5) a um programa que você mesmo escreve.

### Exercício complementar 1: Quantas vezes dá para dividir por 2?

Escreva a função `contar_divisoes(n)` que recebe um número inteiro positivo `n` (o tamanho de uma lista) e devolve **quantas vezes é possível dividi-lo por 2** até o resultado ficar **menor que 1**.

Por exemplo: `contar_divisoes(7)` devolve `3` (7 → 3,5 → 1,75 → 0,875) e `contar_divisoes(1000)` devolve `10`.

Depois, confira que esse número é o **máximo de comparações** da busca binária: use `busca_binaria_comparacoes` (exercício 3) para procurar um item ausente e maior que todos (por exemplo, `5000`) em `list(range(1000))`.

---

## 10. Exercícios do tema complementar: `datetime`

Os exercícios abaixo usam o que foi visto na seção 7. Use as funções `busca_linear` e `busca_binaria` da aula.

### Exercício do tema complementar 1: Cronometrando as buscas

Escreva uma função `cronometrar_buscas(tamanho)` que cria a lista `list(range(tamanho))`, mede o tempo da busca linear e o da busca binária para procurar o item `-1` (que não existe) e exibe os dois tempos em **segundos**, com 6 casas decimais (use `.total_seconds()`). Teste com `tamanho = 1000000`.

### Exercício do tema complementar 2: Tempo x tamanho da lista

Escreva uma função `comparar_tempos_por_tamanho(tamanhos)` que recebe uma lista de tamanhos (por exemplo, `[1000, 10000, 100000, 1000000]`) e exibe uma **tabela** com o tamanho, o tempo da busca linear e o tempo da busca binária de cada um. Depois, observe: quando o tamanho da lista é multiplicado por 10, o que acontece com cada tempo?

---

## 11. Evolução do projeto de fundo: E-commerce simples

Assim como nas Aulas 08 a 12, a Aula 13 apenas **adiciona** funcionalidades novas ao projeto, sem alterar nenhuma função que já existia.

Hoje, quando precisamos encontrar um produto **pelo nome** (como em `atualizar_estoque` e `vender_produto`), percorremos o catálogo um item por vez: é uma busca linear. O catálogo é um dicionário cuja chave é o **código** do produto, e o nome fica dentro dos dados, então o Python não consegue "pular" direto para um nome. Para usar a busca binária por nome precisamos de uma **lista ordenada pelo nome**: um **índice**, como o índice remissivo de um livro, que guarda o nome e diz onde está o produto (o código dele no catálogo).

Cada item do índice é uma lista `[nome, codigo]`, como em uma matriz:

```
[
    ["boné preto", "BON10"],
    ["camiseta azul", "CAM13"],
    ["mochila cinza", "MOC13"],
    ["tênis runner", "TÊN12"],
]
```

O nome fica em minúsculas (assim "Boné Preto" e "boné preto" são considerados o mesmo produto) e a lista fica em **ordem alfabética**.

### Enunciado

1. Crie a função `gerar_indice_por_nome`, que recebe o catálogo e devolve o índice descrito acima: uma lista de listas `[nome_em_minusculas, codigo]`, em ordem alfabética pelo nome. Use a função `sorted()` do Python para ordenar.
2. Crie a função `buscar_produto_por_nome_linear`, que recebe o índice e o nome procurado e faz uma **busca linear** no índice. Ela devolve um **dicionário** com duas chaves: `"codigo"` (o código do produto, ou `None` se não existir) e `"comparacoes"` (quantas comparações foram feitas). O nome informado deve ser normalizado (`.strip().lower()`).
3. Crie a função `buscar_produto_por_nome_binaria`, com os mesmos parâmetros e o mesmo formato de retorno, mas usando **busca binária**.
4. Na demonstração, procure um produto que existe e um que não existe com as duas funções e compare as comparações. Depois, repita a comparação em um índice com 100 mil produtos.

Por que devolver um dicionário e não só o código? Porque queremos duas informações: **o que foi achado** e **quanto custou achar**. Assim, o programa mostra na prática a diferença entre as duas buscas.

Atenção: o índice é uma **cópia ordenada** dos nomes. Se um produto novo for cadastrado depois de gerado o índice, ele **não aparece** nele: é preciso gerar o índice de novo (mais um motivo para ordenar uma vez e buscar muitas, como visto na seção 5).